# CSE251: System Programming

## Appendix of HW #1

Seongil Wi

Department of Computer Science and Engineering

# Overview of Storing Data and Printing Output

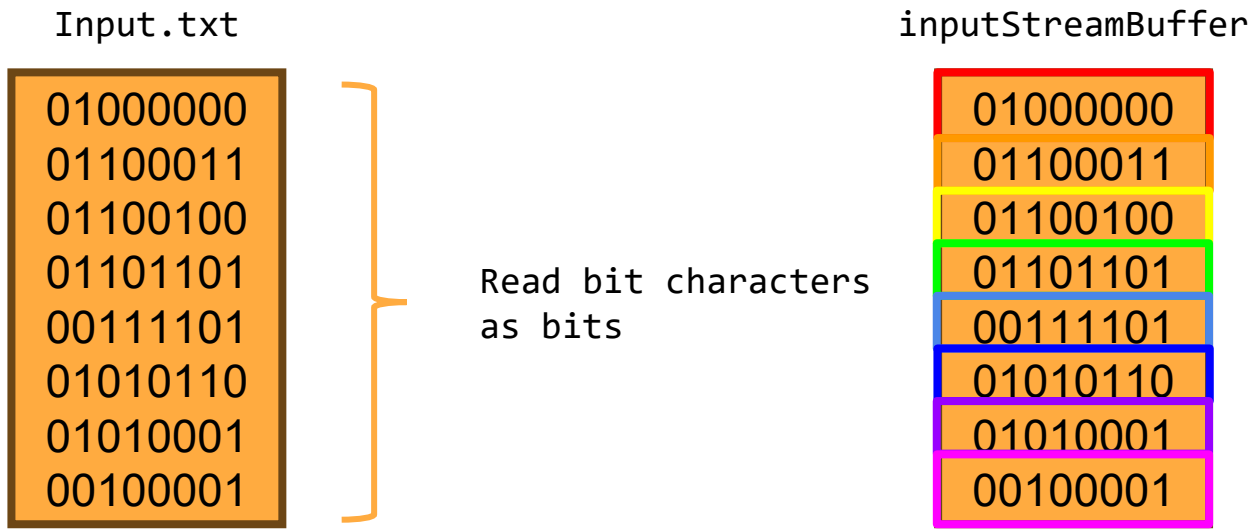
- Clarifying Output Format
  - The input stored in big endian format is converted into little endian format for memory storage, and then outputted according to each data type.
- Explanation Using an Example
  - The first line of the input example, "01000000011000110110010001101101001111010101011001010001001 00001“, will be used to illustrate the process.

Input.txt

```
01000000
01100011
01100100
01101101
00111101
01010110
01010001
00100001
```

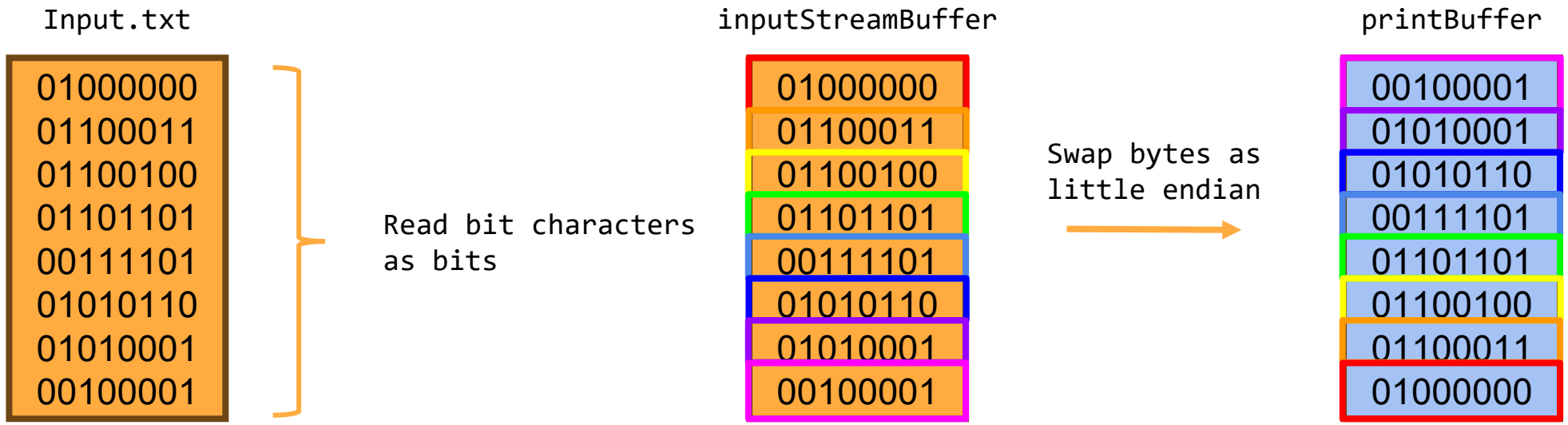
# Input Processing

- Converting Bit Characters to Bytes
  - Group bit characters into 8-bit (1-byte) units and convert them to an unsigned char array.
  - Example: “01000000” → 0x40 (ASCII for ‘@’).
  - Converted array example: “40 63 64 6d 3d 56 51 21”.



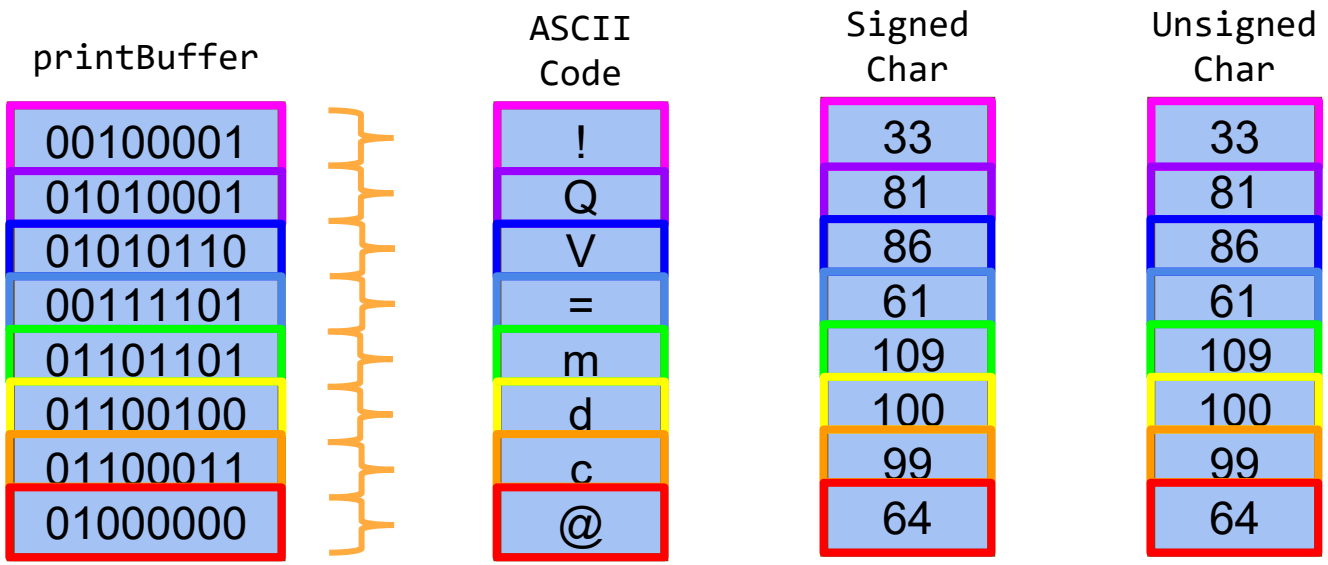
# Byte Array Order Transformation

- Order Transformation Before Copying to Final Buffer
  - Bytes in inputStreamBuffer are reversed in order before being copied to the printBuffer.
  - Final memory storage format: {21 51 56 3d 6d 64 63 40 }.



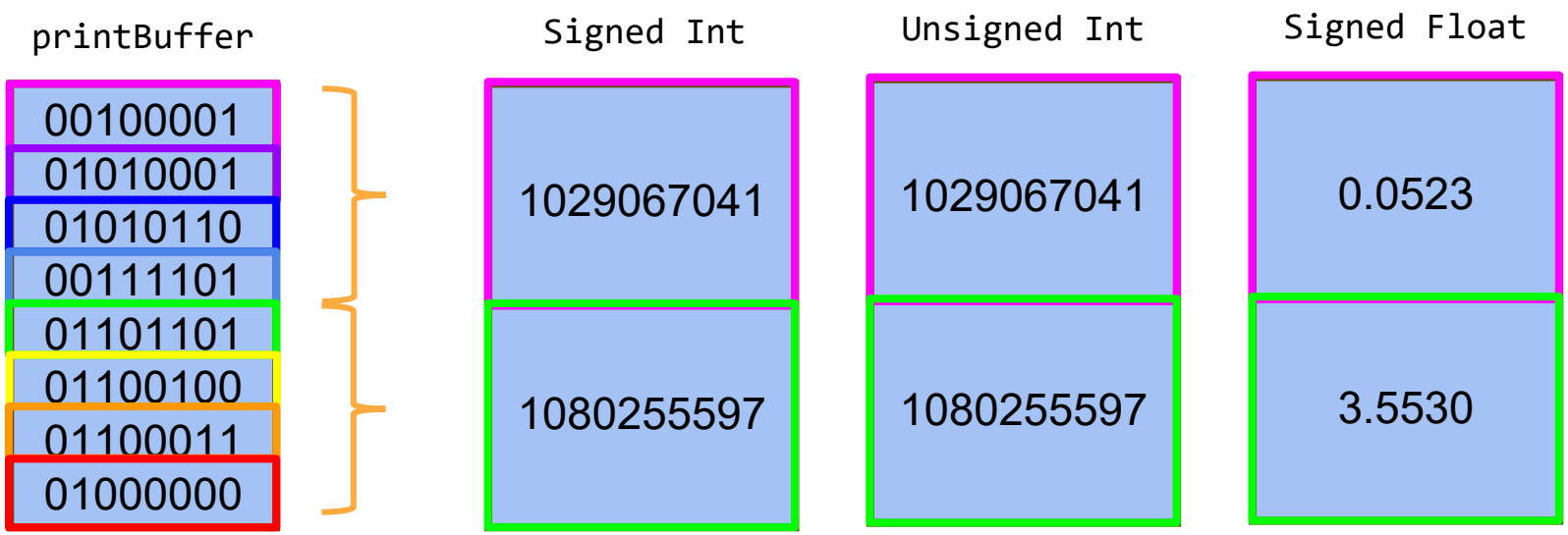
# Output Process - ASCII and Characters

- ASCII Code Output
  - Bytes in printBuffer are interpreted and output as ASCII characters.
  - Valid ASCII characters are printed as is, others are printed as '.'.
- Signed Char Output
  - Bytes are interpreted as signed char and output as decimal numbers.
- Unsigned Char Output
  - Bytes are interpreted as unsigned char and output as decimal numbers.



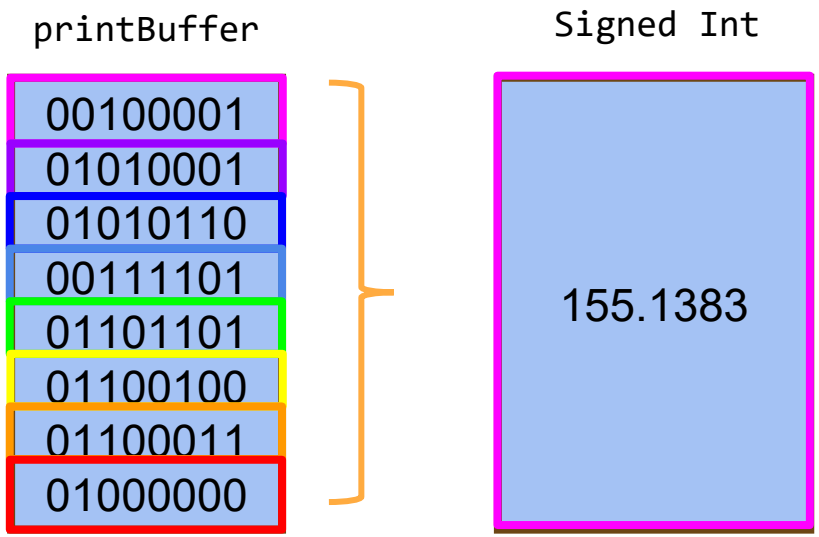
# Output Process - Integers and Floating Points

- Signed and Unsigned Int Output
  - Bytes are grouped into 4-byte units and interpreted/output as signed and unsigned int.
  - Examples: 1029067041 (0x3d565121), 10802559 (0x4063646d).
- Signed Float Output
  - 4-byte units are interpreted and output as float.
  - Examples: 0.0523, 3.5530.



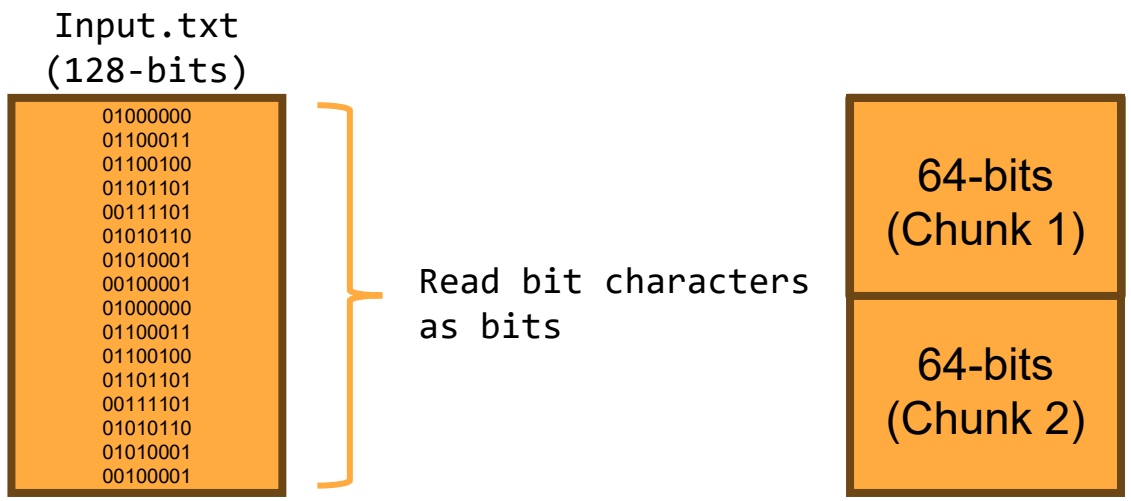
# Output Process - Double

- Signed Double Output
  - 8-byte units are interpreted and output as doubles.
  - Example: 155.1383 (0x4063646d3d565121).



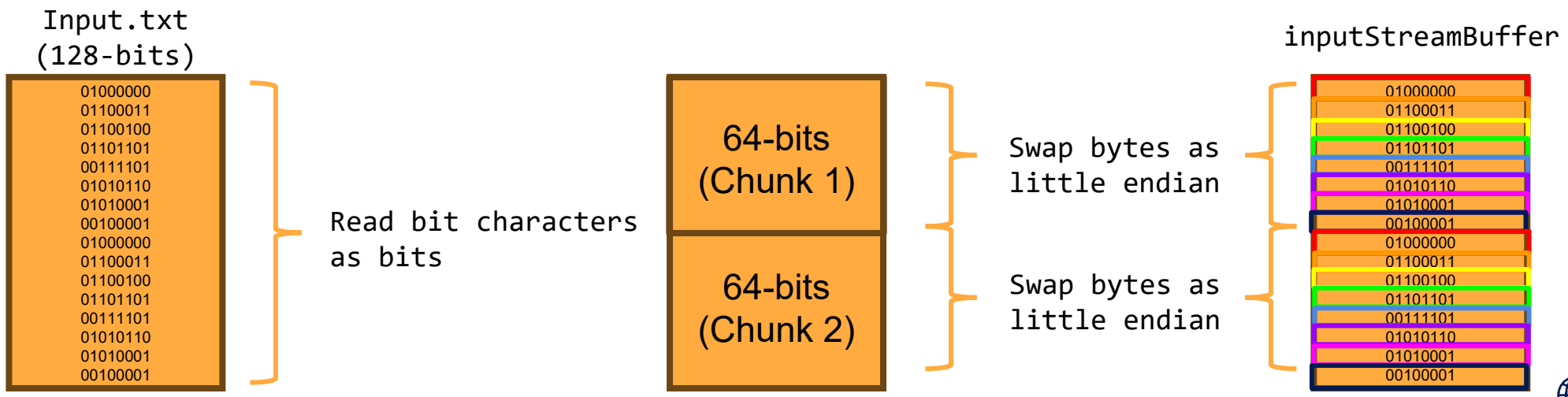
# Details on Little Endian Conversion

- Little Endian Conversion in 64-bit Units
  - The conversion from big endian to little endian occurs in 64-bit chunks.
- Example with 128-bit Input
  - A 128-bit input is divided into two 64-bit chunks.



# Details on Little Endian Conversion

- Byte Order Transformation Within Each Chunk
  - Within each chunk, the order of bytes is reversed for little endian conversion, as illustrated in previous slides. However, the order between the chunks does not changed.
- Maintaining Chunk Order
  - Although the byte order within each 64-bit chunk is reversed due to little endian conversion, the sequence of Chunk 1 and Chunk 2 remains the same. This is because little endian conversion affects the byte order within specific-sized formats but does not alter the order of these formats relative to each other.



# How to test with *random_hex_generator*

- Running the Command
  - Execute the random_hex_generator tool by using the command `./random_hex_generator`
- Utilizing Output for Input
  - Use the last line's content, starting with "Bits:", from the output of `random_hex_generator` as input for your assignment program.

```
ehdrb01  ~/system_programming_hw1  ./random_hex_generator
Random 8-byte Hex Value: 3f5f785d4221524c
Float (Front 4 Bytes): 40.330368
Float (Back 4 Bytes): 0.872930
Double: 0.001921
Integer (Front 4 Bytes): 1109480012
Integer (Back 4 Bytes): 1063221341
ASCII: L R ! B ] x   ?
Bits: 0011111101011111011110000101110101000010001000010101001001001100
```

# Preparing the Input File

- Collecting Bit Contents
  - Run the `random_hex_generator` once or multiple times to collect various “Bits” content and save these in an `input.txt` file.
- Executing the Assignment Program
  - Run your assignment program using the prepared `input.txt` as the input source.

```
ehdrb01  ~/system_programming_hw1  ./random_hex_generator
Random 8-byte Hex Value: 3f5f785d4221524c
Float (Front 4 Bytes): 40.330368
Float (Back 4 Bytes): 0.872930
Double: 0.001921
Integer (Front 4 Bytes): 1109480012
Integer (Back 4 Bytes): 1063221341
ASCII: L R ! B ] x   ?
Bits: 0011111101011111011110000101110101000010001000010101001001001100
ehdrb01  ~/system_programming_hw1  ./random_hex_generator
Random 8-byte Hex Value: 404b79093c554e6d
Float (Front 4 Bytes): 0.013019
Float (Back 4 Bytes): 3.179262
Double: 54.945594
Integer (Front 4 Bytes): 1012223597
Integer (Back 4 Bytes): 1078688009
ASCII: m N U <   y K @
Bits: 0100000001001011011110010000100100111100010101010100111001101101
ehdrb01  ~/system_programming_hw1
```

```
Ubuntu
~
~
~
~
~
~
"input.txt" 0L, 0B
```

```
Ubuntu
0011111101011111011110000101110101000010001000010101001001001100
0100000001001011011110010000100100111100010101010100111001101101
~
~
~
```

# Preparing the Input File

- Analyzing Program Output
  - Compare the results of your assignment program with the output of the `random_hex_generator`.
  - Note that the `random_hex_generator` does not limit decimal places, so consider rounding off the results beyond 4 digits after the decimal point.
- Debugging with a Debugger
  - You can debug your assignment program by comparing the hex values loaded into memory with the “Random 8-byte Hex Value:” from the `random_hex_generator` using debuggers like gdb or lldb.

```
ehdrb01  ~/system_programming_hw1  ./random_hex_generator
Random 8-byte Hex Value: 3f5f785d4221524c
Float (Front 4 Bytes): 40.330368
Float (Back 4 Bytes): 0.872930
Double: 0.001921
Integer (Front 4 Bytes): 1109480012
Integer (Back 4 Bytes): 1063221341
ASCII: L R ! B ] x _ ?
Bits: 0011111101011110111100001011101010000100010000101010010010011 00
ehdrb01  ~/system_programming_hw1  ./random_hex_generator
Random 8-byte Hex Value: 404b79093c554e6d
Float (Front 4 Bytes): 0.013019
Float (Back 4 Bytes): 3.179262
Double: 54.945594
Integer (Front 4 Bytes): 1012223597
Integer (Back 4 Bytes): 1078688009
ASCII: m N U <   y K @
Bits: 0100000001001011011110010000100100111100010101010100111001101101
ehdrb01  ~/system_programming_hw1
```

```
Ubuntu
[*] Signed Char: 76 82 33 66 93 120 95 63 109 78 85 60 9 121 75 64 76 82 33 66 93 120 95 63
[*] ASCII Codes: L R ! B ] x _ ? m N U <         y K @ L R ! B ] x _ ?
[*] Unsigned Char: 76 82 33 66 93 120 95 63 109 78 85 60 9 121 75 64 76 82 33 66 93 120 95 63
[*] Signed Int: 1109480012 1063221341 1012223597 1078688009 1109480012 1063221341
[*] Unsigned Int: 1109480012 1063221341 1012223597 1078688009 1109480012 1063221341
[*] Signed Float: 40.3304 0.8729 0.0130 3.1793 40.3304 0.8729
[*] Signed Double: 0.0019 54.9456 0.0019
```

# Makefile

# Makefile

- Makefiles are a simple way to organize code compilation.
- Makefile is a script used to automate the building, testing, and deployment of software projects.
- Makefile contains a set of instructions (called “rules”) that specify how to compile and link code files, as well as how to run tests and generate executable files.

# Example Code

```c
#include “hello.h”

int main() {
    printhello();
    return 0;
}
```

hellomake.c

```c
#include <stdio.h>
#include “hello.h”

void printhello() {
    printf(“Hello\n”);
    return;
}
```

hellofunc.c

```c
void printhello();
```

hello.h

# Example Code

- gcc -c hellofunc.c
- gcc -c hellomake.c
- gcc -o hellomake hellomake.o hellofunc.o

# Makefile 1

```
hellomake: hellomake.c hellofunc.c
        gcc -o hellomake hellomake.c hellofunc.c -I.
```

```
ehdrb01 ~/system_programming_hw1/makefile ls
Makefile  hello.h  hellofunc.c  hellomake.c
ehdrb01 ~/system_programming_hw1/makefile make
gcc -o hellomake hellomake.c hellofunc.c -I.
ehdrb01 ~/system_programming_hw1/makefile ls
Makefile  hello.h  hellofunc.c  hellomake  hellomake.c
ehdrb01 ~/system_programming_hw1/makefile
```

# Makefile 2

```
CC=gcc
CFLAGS = -I.
hellomake: hellomake.c hellofunc.c
```

```
ehdrb01 ~/system_programming_hw1/makefile ls
Makefile  hello.h  hellofunc.c  hellomake.c
ehdrb01 ~/system_programming_hw1/makefile make
gcc -I.    hellomake.c hellofunc.c   -o hellomake
ehdrb01 ~/system_programming_hw1/makefile ls
Makefile  hello.h  hellofunc.c  hellomake  hellomake.c
ehdrb01 ~/system_programming_hw1/makefile
```

# Makefile 3

```
CC=gcc
CFLAGS = -I.
DEPS = hellomake.h
%.o: %.c $(DEPS)
hellomake: hellomake.o hellofunc.o
clean:
        -rm -rf *.o core *.core
```

```
ehdrb01 ~/system_programming_hw1/makefile ls
Makefile  hello.h  hellofunc.c  hellomake.c
ehdrb01 ~/system_programming_hw1/makefile make
gcc -I.   -c -o hellomake.o hellomake.c
gcc -I.   -c -o hellofunc.o hellofunc.c
gcc   hellomake.o hellofunc.o   -o hellomake
ehdrb01 ~/system_programming_hw1/makefile ls
Makefile  hello.h  hellofunc.c  hellofunc.o  hellomake  hellomake.c  hellomake.o
ehdrb01 ~/system_programming_hw1/makefile make clean
rm -rf *.o core *.core
ehdrb01 ~/system_programming_hw1/makefile ls
Makefile  hello.h  hellofunc.c  hellomake  hellomake.c
ehdrb01 ~/system_programming_hw1/makefile
```

# Makefile References

- GNU Make Manual - https://www.gnu.org/software/make/manual/make.html
- Tutorialspoint - https://www.tutorialspoint.com/makefile/index.htm
- Makefile Tutorial by Derek Wyatt - https://makefiletutorial.com/
- Peter Miller's Recursive Make Considered Harmful - http://aegis.sourceforge.net/auug97.pdf

# File I/O

# File I/O

- `FILE *fopen(const char *filename, const char *mode);`
  - It opens a file and returns a pointer to a `FILE` object.
  - The `filename` specifies the name of the file to open.
  - The `mode` determines how the file is accessed.
- `char *fgets(char *str, int n, FILE *stream);`
  - It reads a line from a file or input stream.
  - It reads up to `n-1` characters and stores them in `str`.
  - It stops reading when a newline character or EOR is encountered.
  - It returns `str` on success, or `NULL` on error

# File I/O

- `int fprintf(FILE *stream, const char *format, ...);`
  - It writes formatted output to a file stream.
  - It is similar to `printf`, but the output is written to the specified stream.
  - It returns the number of characters written, or a negative value if an error occurs.
- `void free(void *ptr);`
  - It releases dynamically allocated memory back to the system.
  - The memory must have been allocated using `malloc, calloc, or realloc.`
  - After calling `free`, the pointer should not be used again unless reassigned.

# Dynamic Allocator

# The Standard Allocator

- The standard allocator provides a convenient method to:
  - Allocate memory on demand
  - Release memory when it is no longer needed
- The UNIX system calls for memory management either:
  - require the application to do extra bookkeeping work, or
  - do not reliably allow for releasing memory.
- In particular, the user need not track allocation sizes when using the standard allocator.

# The Standard Allocator Interface

There are three allocation functions in the standard allocator:

- `void *malloc(size_t size);`
  - Allocates size bytes of memory.
- `void *calloc(size_t nmemb, size_t size);`
  - Allocates an array of nmemb elements of size bytes each, then zeroes the entire array.
- `void *realloc(void *ptr, size_t size);`
  - Behaves like malloc() if ptr is NULL, otherwise adjusts the allocation of ptr to be size bytes if possible. If this is not possible, it creates a new allocation of size bytes and copies the contents of ptr into the new allocation.

# Freeing Memory from the Standard Allocator

Any allocation made by the standard allocator can be freed with free().

- `void free(void *ptr);`
  - This will return the freed memory to the heap.

Freed memory may be used again for future allocations.

# Allocation Sizes

Note that `free()` and `realloc()` must both know allocation sizes.

- `free()` must return memory to the heap
- `realloc()` might return memory to the heap, might copy memory, or might adjust an existing allocation size.

Note also that the allocator-returned pointers:

- Allow the user to use memory starting immediately at the pointer.
- Don’t return any other user-visible metadata.

This dictates that object size is stored *somewhere else*.